

多继承

多继承的需要性

- 对于下面的两个类A和B：

```
class A
{
    int m;
public:
    void fa();
};
class B
{
    int n;
public:
    void fb();
};
```

- 如何定义一个类C，它包含A和B的所有成员，另外还拥有新的数据成员r和成员函数fc？

■ 用单继承实现

```
class C: public A
{   int n,r; //把B类中的n复制过来
    public:
        void fb(); //把B类中的fb复制过来
        void fc();
};
```

• 或者

```
class C: public B
{   int m,r; //把A类中的m复制过来
    public:
        void fa(); //把A类中的fa复制过来
        void fc();
};
```

■ 不足之处：

- 概念混乱：导致A和B之间增加了层次关系。
- 易造成不一致：A中的m、fa（或B中的n、fb）与C中的m、fa（或n、fb）。
- 不能完全实现子类型：只有A和B中某一类对象能被C类对象替代。

- 用成员对象实现

```
class C
{
    A a;
    B b;
    int r;
public:
    void fa() { a.fa(); }
    void fb() { b.fb(); }
    void fc();
};
```

- 不足之处：

- 不能实现子类型：程序中需要A和B类对象的地方都不能用C类对象去替代。

■ 用多继承实现

```
class C: public A, public B
{ int r;
  public:
    void fc();
};
```

- 多继承增强了语言的表达能力，它使得语言能够自然、方便地描述问题领域中的存在于对象类之间的多继承关系。

多继承的类定义

- **多继承**是指派生类可以有一个以上的直接基类。多继承的派生类定义格式为：

```
class <派生类名> : [<继承方式>] <基类名1>,  
                  [<继承方式>] <基类名2>, ...  
{ <成员说明表>  
};
```

- 继承方式及访问控制的规定同单继承。
- 派生类拥有所有基类的所有成员。
- 基类的声明次序决定：
 - 对基类数据成员的存储安排。
 - 对基类构造函数/析构函数的调用次序

```

class A
{
    int m;
public:
    void fa();
};
class B
{
    int n;
public:
    void fb();
};

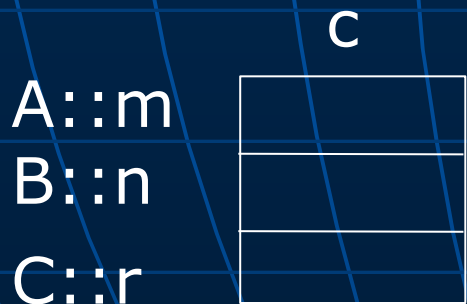
```

```

class C: public A, public B
{
    int r;
public:
    void fc();
};
.....
C c;
c.fa(); //OK
c.fb(); //OK
c.fc(); //OK

```

- 对象c的内存空间布局是：



- 构造函数的执行次序是：

A()、B()、C()

(A()和B()实际是在C()的成员初始化表中调用。)

- 需要A类或B类对象的地方可以用C类对象去替代。
- 把多继承派生类对象的地址赋给基类的指针时将会自动进行地址调整。例如，对于前面的三个类：

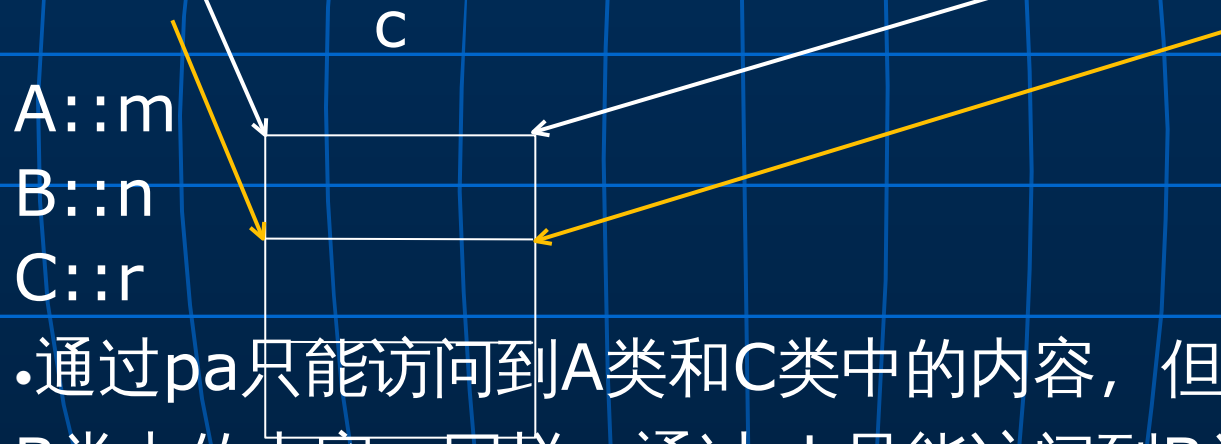
```
C c;
```

```
A *pa=&c;
```

```
B *pb=&c;
```

```
c.fa();-->A::fa(this);
```

```
c.fb();-->B::fb(this);
```



•通过pa只能访问到A类和C类中的内容，但不会访问到B类中的内容。同样，通过pb只能访问到B类和C类中的内容，但不会访问到A类中的内容。（通过pa或pb访问C类中的内容一般要通过虚函数动态绑定来实现）

多继承带来的问题

- 多继承使得语言特征复杂化、加大了编译程序的难度以及使得消息绑定复杂化等，从而给正确使用多继承带来困难。因此，有些面向对象语言（如 Smalltalk、Java等）放弃了多继承机制。
- 多继承带来的两个主要问题：
 - 名冲突问题
 - 重复继承问题

多继承中的名冲突问题

```
class A
{
    .....
    public:
        void f();
        void g();
};
```

```
class B
{
    .....
    public:
        void f();
        void h();
};
```

```
class C: public A, public B
{
    .....
    public:
        void func()
        {
            f(); //Error, 是A的f, 还是B的f?
        }
};

.....
C c;
c.f(); //Error, 是A的f, 还是B的f?
```

- 解决名冲突的办法是：基类名受限

```
class C: public A, public B
```

```
{
```

```
.....
```

```
public:
```

```
void func()
```

```
{    A::f(); //OK, 调用A的f。
```

```
      B::f(); //OK, 调用B的f。
```

```
}
```

```
};
```

```
.....
```

```
C c;
```

```
c.A::f(); //OK, 调用A的f。
```

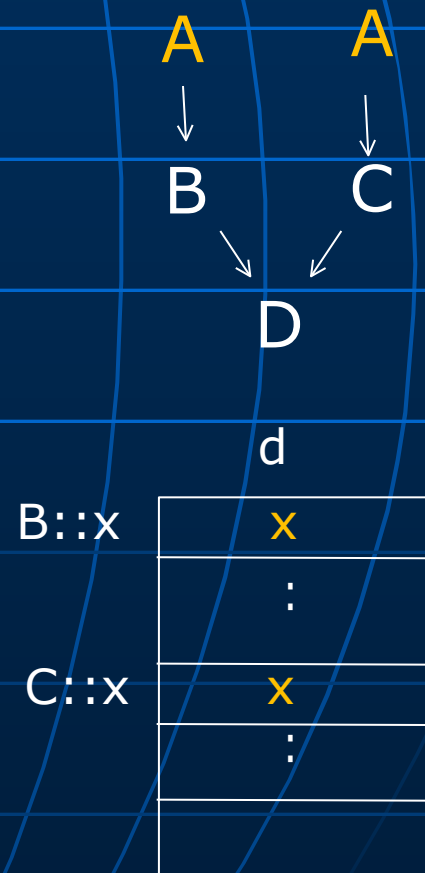
```
c.B::f(); //OK, 调用B的f。
```

重复继承问题 - - 虚基类

- 下面的类D从类A继承两次，称为重复继承：

```
class A
{ int x;
  .....
};
class B: public A { ..... };
class C: public A { ..... };
class D: public B, public C { ..... };
D d;
```

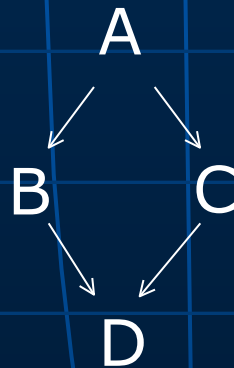
- 上面D类的对象d将包含两个x：
 - B::x和C::x



- 如果要求类D中只有一个x，则应把A定义为B和C的**虚基类**：

```
class A { int x; .....};  
class B: virtual public A {.....};  
class C: virtual public A {.....};  
class D: public B, public C {.....};  
D d;
```

- 这样，D类的对象d就只有一个x了。



虚基类构造函数的调用

- 创建包含虚基类的类对象时：
 - 虚基类的构造函数由该类的构造函数直接调用。
 - 虚基类的构造函数优先非虚基类的构造函数执行。

```
class A
{ int x;
public:
  A(int i) { x = i; }
};
class B: virtual public A //包含虚基类A
{ int y;
public:
  B(int i): A(1) { y = i; }
};
class C: virtual public A //包含虚基类A
{ int z;
public:
  C(int i): A(2) { z = i; }
};
```

```

class D: public B, public C //包含虚基类A
{
    int m;
public:
    D(int i, int j, int k): B(i), C(j), A(3) { m = k; }
};
class E: public D //包含虚基类A
{
    int n;
public:
    E(int i, int j, int k, int l): D(i,j,k), A(4) { n = l; }
};
.....

```

D d(1,2,3); //这里， A的构造函数由D调用， d.x初始化为3。

- 调用的构造函数及它们的执行次序是：
A(3)、B(1)、C(2)、D(1,2,3)

E e(1,2,3,4); //这里， A的构造函数由E调用， e.x初始化为4。

- 调用的构造函数及它们的执行次序是：
A(4)、B(1)、C(2)、D(1,2,3)、E(1,2,3,4)

虚基类的实现

非虚基类

```
class A
{
    int x;
};
class B: public A
{
    int y;
    void fb() { x, y } //this->x, this->y
};
class C: public A
{
    int z;
    void fc() { x, z } //this->x, this->z
};
class D: public B, public C
{
    int m;
};
```

D d; d.fb(); d.fc();

x:

y:

x:

z:

m:

sizeof(d) == 5*sizeof(int)!

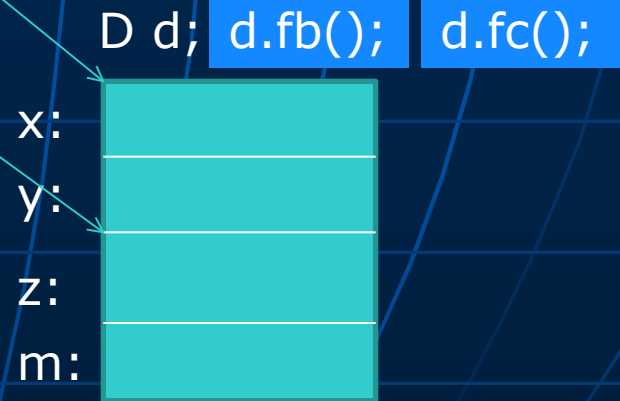
虚基类的一种实现

(简单去掉一个x)

可行吗？

```
class A
{
    int x;
};
class B: virtual public A
{
    int y;
    void fb() { x, y } //this->x, this->y
};
class C: virtual public A
{
    int z;
    void fc() { x, z } //this->x, this->z
};
class D: public B, public C
{
    int m;
};
```

访问不到x！



虚基类的实现

(x移到最后, 在原来x的位置
存储一个x的偏移量指针)

```
class A
{
    int x;
};
class B: virtual public A
{
    int y;
    void fb() { x, y } //this->...x, this->y
};
class C: virtual public A
{
    int z;
    void fc() { x, z } //this->...x, this->z
};
class D: public B, public C
{
    int m;
};
```

虚基类表:
vtable

$\text{sizeof}(d) == 6 * \text{sizeof}(\text{int})!$

